

MCGRAD: Multicalibration at Web Scale

Niek Tax
Meta Platforms, Inc.
London, United Kingdom
niek@meta.com

Lorenzo Perini
Meta Platforms, Inc.
London, United Kingdom
lorenzoperini@meta.com

Fridolin Linder
Meta Platforms, Inc.
Munich, Germany
flinder@meta.com

Daniel Haimovich
Meta Platforms, Inc.
London, United Kingdom
danielha@meta.com

Dima Karamshuk
Meta Platforms, Inc.
London, United Kingdom
karamshuk@meta.com

Nastaran Okati
Max Planck Institute for Software
Systems
Kaiserslautern, Germany
nastaran@mpi-sws.org

Milan Vojnovic
Meta Platforms, Inc.
London, United Kingdom
The London School of Economics and
Political Science
London, United Kingdom
m.vojnovic@lse.ac.uk

Pavlos Athanasios
Apostolopoulos
Meta Platforms, Inc.
Menlo Park, California, USA
pavlosapost@meta.com

Abstract

We propose MCGRAD, a novel and scalable multicalibration algorithm. Multicalibration - calibration in subgroups of the data - is an important property for the performance of machine learning-based systems. Existing multicalibration methods have thus far received limited traction in industry. We argue that this is because existing methods (1) require such subgroups to be manually specified, which ML practitioners often struggle with, (2) are not scalable, or (3) may harm other notions of model performance such as log loss and Area Under the Precision-Recall Curve (PRAUC). MCGRAD does not require explicit specification of protected groups, is scalable, and often improves other ML evaluation metrics instead of harming them. MCGRAD has been in production at **Meta**, and is now part of hundreds of production models. We present results from these deployments as well as results on public datasets. We provide an open source implementation of MCGRAD at <https://github.com/facebookincubator/MCGrad>.

Keywords

Multicalibration, Calibration, Uncertainty Quantification, Fairness

1 Introduction

A machine learning model is said to be *calibrated* when its predictions match the true outcome frequencies [2, 38, 55]. The importance of calibration to ML-based systems has been widely recognized [24, 33, 37], including in web applications such as content ranking [39, 47], recommender systems [11, 23, 56], digital advertising [7, 9, 17] and content moderation [35, 40, 58]. Calibration is necessary for such systems to make optimal decisions without under- or overestimating risk or opportunity.

Multicalibration is a more powerful property that extends the concept of calibration to ensure that predictors are simultaneously calibrated on various (potentially overlapping) groups [28, 31, 45, 48]. Often, multicalibration is applied as a post-processing step to

fix a base classifier’s predictions. While it originated in the study of algorithmic fairness, its significance extends far beyond it. A growing body of work shows that multicalibration can increase model performance and robustness in many circumstances, ranging from out-of-distribution prediction to confidence scoring of LLMs [14, 16, 22, 34, 59, 60].

Despite its potential, multicalibration has received limited traction in industry. We argue that this is due to three main reasons. **First**, existing multicalibration methods require *protected groups* to be *manually defined* [30]: not only do users need to specify the covariates that require protection (e.g., user age and user country), but also they must define concrete protected groups through binary membership indicators, such as “is an adult in the US?”. This poses a significant challenge to real-world application of multicalibration because practitioners (a) may not have established the precise set of protected groups, (b) may be subject to changes to the definition of protected groups over time thus requiring ongoing effort and change management (e.g., due to changes related to legal context, policy frameworks, ethical considerations, or otherwise), or (c) may only seek an overall performance improvement without prioritizing specific groups for protection. The difficulties defining protected groups become even more pronounced when the set of features that we want to protect is large. **Second**, existing methods lack the ability to scale to large datasets and/or many protected groups, which makes them hard to deploy in production. For instance, Hebert-Johnson et al. [28]’s algorithm scales at least linearly in time and memory with the number of groups, being potentially inefficient when a large set of groups is specified. To be deployed at web scale, multicalibration methods must be highly optimized so as to not introduce significant computational requirements at training or inference time. **Third**, existing multicalibration methods lack guardrails to be safely deployed, and risk harming model performance (e.g., due to overfitting), which may prevent practitioners from deploying them altogether.

In this paper, we introduce MCGRAD (MultiCalibration Gradient Boosting) a novel multicalibration algorithm deployed in production that is lightweight and safe to use. MCGRAD requires only the specification of a set of protected features, rather than predefined protected groups. It then identifies miscalibrated regions within this feature space, and calibrates a base classifier’s predictions over all groups that can be defined based on these features. On a high level, MCGRAD uses a Gradient Boosted Decision Tree (GBDT) algorithm recursively, such that the result converges to a multicalibrated predictor (Section 3.1). By using a highly optimized GBDT implementation, like LightGBM [32], MCGRAD inherits scalability and regularization (Section 3.2). Finally, it employs an early stopping procedure to avoid overfitting, ensuring that model performance is not harmed (Section 3.3).

Empirically, on benchmark datasets, we show that MCGRAD improves the base predictor’s outputs in terms of multicalibration, calibration, and even predictive performance (Section 4). We complement this analysis by describing the impact of MCGRAD on real-world data: MCGRAD has been used in production at **Meta** on hundreds of machine learning models, and in total, generates over a million multicalibrated real-time predictions per second (Section 5). MCGRAD successfully addresses all three previous challenges and, to the best of our knowledge, this is the largest-scale adoption of multicalibration in production. Our deployment results show significant impact: on 24 of 27 models tested via A/B testing on our Looper platform [42], MCGRAD significantly outperformed Platt scaling, leading to promotion of the MCGRAD-calibrated variants to production. Additionally, across 120+ models on our internal ML platform, MCGRAD improved log loss for 88.7% of models, PRAUC for 76.7%, and Expected Calibration Error for 86.0%. With these real-world results, we add to the growing body of evidence that shows that multicalibration can significantly contribute to model performance. We also provide valuable practical learnings from applying multicalibration in industry (Section 6), and link multicalibration to related areas (Section 7).

2 Background on (Multi)Calibration

Let $\mathcal{X} = \mathbb{R}^d$ be a d -dimensional input space and $\mathcal{Y} = \{0, 1\}$ the binary target space. Let $D = \{(x_i, y_i)\}_{i=1}^n \sim p(X, Y)$ be the dataset with n i.i.d. samples, where the random variables X, Y represent, respectively, a d -dimensional feature vector and the target. A probabilistic predictor f is a map $f: \mathcal{X} \rightarrow [0, 1]$, which assigns an instance $x \sim p(X)$ to an estimate of its class conditional probability $p(Y = 1 | X = x)$. For any probabilistic predictor f , we denote with F its logit (referred to as the predictor), i.e., $f(x) = \mu(F(x))$, where $\mu(F) = 1/(1 + e^{-F})$ is the sigmoid function. We assume a base probabilistic predictor f_0 is given; this predictor is the one targeted for calibration via a post-processing procedure.

Let $\mathcal{L}: \mathbb{R} \times \mathcal{Y} \rightarrow \mathbb{R}$ be the log loss (negative log-likelihood), i.e., for any $(x, y) \in \mathcal{X} \times \mathcal{Y}$ and any predictor F ,

$$\mathcal{L}(F(x), y) = -[y \log(\mu(F(x))) + (1 - y) \log(1 - \mu(F(x)))].$$

We use the symbol $\mathbb{E}$ for the expectation with respect to an arbitrary distribution that is clear from the context, and $\mathbb{E}_D$ for the sample mean over D .

Calibration. A predictor f is perfectly calibrated if and only if $\mathbb{P}(Y = 1 | f(X) = p) = p$, where p is the true underlying probability. Intuitively, for all input pairs $(x, y) \in D$, if f predicts 0.8, we expect that 80% of them have 1 as label. Recently, Tygert [57] introduced the Estimated Cumulative Calibration Error (ECCE), a novel parameter-free metric for measuring calibration that computes the maximum difference between the means of labels and predictions over any score interval:

$$\text{ECCE}(f) = \frac{1}{n} \max_{1 \leq i \leq j \leq n} \left| \sum_{k=i}^j (y_{(k)} - f(x_{(k)})) \right|, \quad (1)$$

where $(x_{(1)}, y_{(1)}), \dots, (x_{(n)}, y_{(n)})$ are the points $\{(x_i, y_i)\}_{i \in D}$ ordered such that $f(x_{(1)}) \leq \dots \leq f(x_{(n)})$.

This metric enjoys useful statistical properties. In particular, it can be standardized by dividing it by the following scaling statistic:¹

$$\sigma(f) = \sqrt{\frac{1}{n^2} \sum_{k=1}^n f(x_k)(1 - f(x_k))}.$$

This gives rise to two ways of interpreting the metric. The absolute scale $\text{ECCE}(f)$ is designed to measure the magnitude of miscalibration. When the data size increases, $n \rightarrow \infty$, $\text{ECCE}(f)$ converges to a true magnitude of miscalibration (0 if f is calibrated and a positive value otherwise). The sigma scale $\text{ECCE}(f)/\sigma(f)$ however converges to a distribution when f is calibrated, and diverges to infinity when it is miscalibrated. The two scales are used to answer different questions. $\text{ECCE}(f)$ is used to answer the question "is the miscalibration large?". $\text{ECCE}(f)/\sigma(f)$ is used to answer the question "is there statistical evidence of miscalibration?". For example, we might have that $\text{ECCE}(f_1) = 0.05 = 20\sigma(f_1)$ and $\text{ECCE}(f_2) = 0.05 = 2\sigma(f_2)$. This would tell us that while the magnitude of miscalibration on the data appears to be similar for f_1, f_2 , the result is statistically significant for f_1 but not for f_2 , which could simply be due to random sampling. See [1, 57] for further discussion.

Multicalibration. Unlike calibration, multicalibration considers the intersection of the *score interval* and the *protected groups*. Let $\mathcal{G} \subset \{g: [0, 1] \rightarrow \{0, 1\}\}$ be the set of *interval membership functions*, i.e. $g(v) = \mathbb{1}_I(v)$ for $v \in [0, 1]$, and any interval $I \subseteq [0, 1]$. Similarly, let $\mathcal{H} \subseteq \{h: \mathcal{X} \rightarrow \{0, 1\}\}$ be a set of *group membership functions* over $\mathcal{X}$, such that a point $x \in \mathcal{X}$ belongs to a group $h \in \mathcal{H}$ if and only if $h(x) = 1$. The groups can be overlapping, i.e., some point $x \in \mathcal{X}$ may belong to multiple groups.

Multicalibration is defined in various ways across the literature [3, 13, 22, 26, 28, 52]. Here, we employ an operational definition of multicalibration:

Definition 2.1 (Multicalibration). A probabilistic predictor f is α -multicalibrated (α -MC) with respect to $\mathcal{H}$ if, for the MC-deviation given as $\Delta_{h,g}(f) := |\mathbb{E}[h(X)g(f(X))(Y - f(X))]|$ and a given scale parameter $\tau_h(f)$:

$$\Delta_{h,g}(f) \leq \alpha \tau_h(f) \quad \forall h \in \mathcal{H}, g \in \mathcal{G}.$$

We say that f is *multicalibrated* if $\alpha = 0$.

¹Notice that $\sigma(f)$ is the standard deviation of the sample mean of prediction labels with prediction probabilities according to f . For a calibrated predictor f , the expected value of $\text{ECCE}(f)/\sigma(f)$ scales as c , for large data size n , with $c = 2\sqrt{2/\pi} \approx 1.6$ [1].

A common choice is to use a uniform bound on the MC-deviation, which is accommodated by choosing the scale parameter $\tau_h(f)$ to be a constant equal to 1. In Appendix A, we show that Definition 2.1, with $\tau_h(f)^2 = \mathbb{E}[h(X)f(X)(1-f(X))]$, naturally matches with Multicalibration Error (MCE) by Guy et al. [25], which quantifies multicalibration error by computing the maximum ECCE over the protected groups with a suitable normalization:

$$\text{MCE}(f) = \max_{h \in \mathcal{H}} \frac{\text{ECCE}_h(f)}{\sigma_h(f)} \quad (2)$$

where $\text{ECCE}_h(f), \sigma_h(f)$ are defined as $\text{ECCE}(f), \sigma(f)$ above but restricted to the group h . Roughly speaking, MCE measures the strongest statistical evidence of miscalibration across protected groups. The unit of the MCE is the same as for ECCE/σ . It can be re-scaled to an absolute measure by multiplying with $\sigma(f)$.

Using a multicalibration metric that is equivalent to the definition of α -MC can help ensure that empirical work is aligned with theory. In Section 4 we use MCE to measure multicalibration, thereby directly estimating the minimal α for which a model is α -MC. To our knowledge, this is the first empirical multicalibration study that achieves this. See Appendix A for further discussion.

3 MCGRAD: A Practical Algorithm for Multicalibration

This paper tackles the following problem:

Given a labeled dataset D , an initial probabilistic predictor f_0 ;

Return a probabilistic predictor f that is α -MC w.r.t. $\mathcal{H}$.

Designing an algorithm for multicalibration without specifying a set of groups is challenging for four reasons. First, the absence of groups requires the multicalibration model to automatically determine the regions of the feature space where the base model is highly miscalibrated. Second, adjusting the model’s output to improve its calibration in a specific region might harm the predictions over other regions, thus increasing the multicalibration error. Third, a post-processing algorithm has to be fast and lightweight to prevent large memory consumption or increased latency of model inference. Fourth, while existing algorithms claim to achieve multicalibration on the training set, they are subject to overfitting issues.

Our proposed algorithm MCGRAD, presented in Algorithm 1, is a lightweight algorithm that multicalibrates a base model without requiring the specification of the groups on which the model has to be calibrated. MCGRAD relies on a key observation: by including the base model’s predictions as a feature, the loss function of the multicalibration algorithm captures the feature values that correspond to miscalibrated initial predictions. In addition, decreasing the loss function implies correcting the base model’s predictions for some regions of the feature space. Because corrections in some regions may negatively impact others, MCGRAD runs multiple rounds, using the previous round’s processed predictions as input. Since GBDTs are regularized using shrinkage (or a step size < 1), a simple rescaling step reduces the number of trees and rounds required, while having a negligible impact on overfitting. Finally, MCGRAD employs early stopping to avoid overfitting.

Algorithm 1 MCGRAD

Input: a probabilistic predictor f_0 , a dataset D ;
 $D_{\text{train}}, D_{\text{valid}} = \text{train_validation_split}(D)$
 $F_0 = \mu^{-1}(f_0)$ # inverse sigmoid transformation
 $\varepsilon_{-1} = +\infty, \varepsilon_0 = \mathbb{E}_{D_{\text{valid}}} [\mathcal{L}(F_0(X), Y)]$ # initialize errors
 $t = 1$ # initialize number of rounds for early stopping
while $\varepsilon_{t-1} - \varepsilon_t > 0$ **do**
 $h_t(x, f_{t-1}(x)) = \text{fit GBDT on } \{(x_i, f_{t-1}(x_i)), y_i\}_{i \in D_{\text{train}}}$
 $\theta_t = \arg \min_{\theta} \mathbb{E}_{D_{\text{train}}} [\mathcal{L}(\theta \cdot (F_{t-1}(X) + h_t(X, f_{t-1}(X))), Y)]$
 $F_t(x) = \theta_t \cdot (F_{t-1}(x) + h_t(x, f_{t-1}(x)))$
 $f_t(x) = \mu(F_t(x))$
 $\varepsilon_{t+1} = \mathbb{E}_{D_{\text{valid}}} [\mathcal{L}(F_t(X), Y)]$
 $t \leftarrow t + 1$
end while
Found $t - 1$ to be the best number of rounds
for s in $[1, \dots, t - 1]$ **do**
 $h_s(x, f_{s-1}) = \text{fit GBDT on } \{(x_i, f_{s-1}(x_i)), y_i\}_{i \in D}$
 $\theta_s = \arg \min_{\theta} \mathbb{E}_D [\mathcal{L}(\theta \cdot (F_{s-1}(X) + h_s(X, f_{s-1}(X))), Y)]$
 $F_s(x) = \theta_s \cdot (F_{s-1}(x) + h_s(x, f_{s-1}(x)))$
 $f_s(x) = \mu(F_s(x))$
end for
 $f(x) = \mu(F_{t-1}(x))$
Output: f ;

3.1 Achieving Multicalibration with T Rounds

Our intuition builds on the following two key insights. First, GBDT returns a solution f that approximately sets the gradient of the loss function $\mathcal{L}$ to zero:

$$\mathbb{E}_D [h(X)(Y - f(X))] = 0 \quad \forall h \in \mathcal{H},$$

where $\mathcal{H}$ is the set of all regression trees over the feature space.

Second, if we augment the feature space X with a single additional feature $f_0(x)$, then the GBDT approximately achieves

$$\mathbb{E}_D [h(X, f_0(X))(Y - f_1(X))] = 0 \quad \forall h \in \mathcal{H}' \quad (3)$$

where $\mathcal{H}' \subset \{h : X \times [0, 1] \rightarrow \mathbb{R}\}$ is the space of trees with an extra real-valued input, and $f_1(x) = \mu(F_0(x) + h_1(x, f_0(x)))$. This creates a bridge between (a) group and interval membership functions and (b) regression trees over the augmented feature space. Strictly speaking, for any $h \in \mathcal{H}, g \in \mathcal{G}$ there exists $h' \in \mathcal{H}'$ such that $h(x)g(f(x)) = h'(x, f(x))$ for $x \in X$.

Perhaps surprisingly, Eq. (3) is equivalent to 0-multicalibration according to Definition 2.1 under the condition $f_1 \equiv f_0$. However, this condition may not always hold: *correcting f_0 for some regions identified by $h(x, f_0(x))$ returns a predictor f_1 that might be miscalibrated on regions identified by its own predictions $h(x, f_1(x))$* . To resolve this, we create a loop over multiple rounds: in each round t , we train a GBDT model f_t using the features x and the previous round’s predictions f_{t-1} , such that, after T rounds, we have

$$\mathbb{E}_D [h(X, f_{T-1}(X))(Y - f_T(X))] = 0 \quad \forall h \in \mathcal{H}^*$$

where $f_T(x) = \mu(F_{T-1}(x) + h_T(x, f_{T-1}(x)))$.

Intuitively, if for some probabilistic predictor f^* , $f_T \rightarrow f^*$ as $T \rightarrow \infty$, then for all $h \in \mathcal{H}^*, \mathbb{E}_D [h(X, f_{T-1}(X))(Y - f_T(X))] \rightarrow \mathbb{E}_D [h(X, f^*(X))(Y - f^*(X))] = 0$ as $T \rightarrow \infty$, which is our desired

condition. This suggests that, for sufficiently large T , the probabilistic predictor f_T should be approximately multicalibrated. See Appendix B for further intuitions and theoretical results.

3.2 Fast Training and Prediction

To be deployed at web scale, there are strict requirements for an algorithm to be efficient both at training and inference time. Several design choices ensure that MCGRAD meets these requirements.

Efficient gradient boosting. First, the algorithm has been designed to rely on a relatively small number of calls to a GBDT, delegating the most compute intensive steps to one of many highly optimized GBDT implementations (e.g. [8, 32]). This differentiates it from existing implementations such as [28]. In our implementation, we use LightGBM.

Rescaling the logits. GBDTs are regularized in multiple ways, including using a step size smaller than 1 for scaling each additional tree. While this helps to avoid overfitting, it results in a predictor which can be improved by rescaling it by a factor slightly greater than 1. As a result, the next round will attempt to apply this rescaling. Since linear rescaling is not easily expressed by decision trees, this may require many trees, ending up requiring unnecessarily many trees to achieve multicalibration.

We introduce a simple rescaling after every round:

$$\theta_t = \arg \min_{\theta} \mathbb{E}_D [\mathcal{L}(\theta \cdot (F_{t-1}(X) + h_t(X, f_{t-1}(X))), Y)]$$

where h_t is obtained with LightGBM. We call the round t 's learned predictor (on the logits) as $F_t = \theta_t(F_{t-1} + h_t)$.

Note that this constant is typically very close to 1 and has a very limited detrimental effect on regularization. Rescaling is not the same as using a step size of 1: it affects the whole sum of trees (i.e., the whole predictor), while the step size only targets each tree sequentially (i.e., each weak learner).

3.3 Preventing Overfitting

Modern GBDT algorithms support various methods for regularization, such as limiting the growth of the trees, the leaf splits, and the number of trees. MCGRAD's recursive structure can give rise to additional overfitting beyond standard GBDTs, which we address in the following ways.

Early stopping. While multiple rounds are required for convergence, they also increase the capacity of the model. With T rounds and M trees in each round, the MCGRAD model is at least as expressive as a tree ensemble with $T \cdot M$ trees. Since the model capacity of tree ensembles is $\Omega(\text{number of trees})$ (see [53]), overfitting is more likely for a large number of rounds.

We solve this problem with a standard early stopping procedure on the number of rounds T . Specifically, we split i.i.d. the dataset D into training (D_{train}) and validation (D_{valid}) sets, and determine the number of rounds T by taking the last round before the expected loss over the validation set increases. That is,

$$T = \min\{t: \mathbb{E}_{D_{valid}} [\mathcal{L}(F_{t+1}(X), Y) - \mathcal{L}(F_t(X), Y)] > 0\} - 1,$$

where the models f_t are obtained using D_{train} instead of D . Using early stopping in a real-world deployed system has a relevant consequence: MCGRAD does not harm the base model's prediction, and,

instead, would select $T = 0$ as optimal number of rounds if the first step decreased the initial performance:

$$\mathbb{E}_{D_{valid}} [\mathcal{L}(F_1(X), Y) - \mathcal{L}(F_0(X), Y)] > 0 \implies T = 0$$

which means that $f_T = f_0$.

Regularizing through the min sum Hessian in leaf. Augmenting the data with the previous round's model necessarily gives rise to regions of the augmented feature space that are particularly prone to overfitting. Consider a leaf of a partially constructed tree in GBDT. The leaf splitting algorithm can choose to split the leaf on values of the previous predictor $f_t(x)$. In this leaf, the left tail $f_t(x) < a$ for some small a will contain only negative labels even if the true distribution may assign a positive probability to a positive label (and analogously for the right tail). In that case the new model can improve the likelihood simply by assigning a very low probability to this tail, as low as zero. Common regularization strategies, like setting the minimum number of samples per leaf, are insufficient to address this scenario. We solve this problem by using a more targeted form of regularization that limits the minimum total Hessian in a leaf, which is offered as one of multiple regularization techniques in LightGBM. The Hessian in a leaf S equals $\sum_{(x,y) \in S} f_t(x)(1 - f_t(x))$ and, as such, is a refined version of the simple sample size rule $|S|$. As the predicted probabilities become close to 0 or 1, the total Hessian becomes smaller and split in multiple leaves can no longer be considered. As shown in our ablation studies (Section 4) this regularization reduces overfitting and improves performance.

4 Benchmark Experimental Analysis

We evaluate MCGRAD on both public and production data. In this section, we present experiments on benchmark datasets and compare MCGRAD against state-of-the-art baselines. In Section 5 we present results of MCGRAD in production. Here, we focus on five research questions:²

- Q1. How does MCGRAD compare to *existing baselines* on unspecified groups?
- Q2. Existing methods require manual specification of protected groups. Does MCGRAD still protect those specified groups, even though they are not specified in MCGRAD?
- Q3. Does MCGRAD benefit from running multiple rounds?
- Q4. What is the effect of *rescaling the logits* and the *regularization through min sum Hessian in leaf* on MCGRAD?
- Q5. How does MCGRAD's computational time compare to existing multicalibration methods?

4.1 Experimental Setup

Data. Our experimental analysis uses 11 datasets (see Table 1). Six binarized prediction tasks are derived from the American Community Survey (ACS) [15, 18] using the folktale package, plus a California-specific income task (ACSIIncomeCA). The remaining five datasets are: UCI Bank Marketing [43] (term deposit prediction), UCI Default of Credit Card Clients [61] (debt default), Home Mortgage Disclosure Act (HMDA) [10] (mortgage acceptance), and

²Code: https://github.com/facebookresearch/mcgrad_multicalibration_at_web_scale.

Dataset	# Samples n	# Features d	Class Distr.	Protected Attributes
MEPS [54]	11079	139	0.169	Race, Gender, Age, Income, Insurance Status
Credit [61]	30000	118	0.221	Gender, Age, Education, Marital Status
Marketing [43]	45211	41	0.116	Age, Marital Status, Education, Occupation
HMDA [10]	114185	89	0.752	Race, Gender, Ethnicity, Age
ACSIIncomeCA [15, 18]	195665	10	0.410	Race, Gender, Age, Education, Income, Employment
ACSMobility [15, 18]	616207	23	0.735	Race, Gender, Age, Education, Income, Employment
ACSPublic [15, 18]	1123374	24	0.293	Race, Gender, Age, Education, Income, Employment
ACSTravel [15, 18]	1458542	19	0.437	Race, Gender, Age, Education, Income, Employment
ACSIIncome [15, 18]	1655429	10	0.370	Race, Gender, Age, Education, Income, Employment
ACSEmploy [15, 18]	3207990	20	0.456	Race, Gender, Age, Education, Income, Employment
ACSHealth [15, 18]	3207990	31	0.150	Race, Gender, Age, Education, Income, Employment

Table 1: Number of samples, features, class distribution ($p(Y = 1)$), and protected attributes for each benchmark dataset.

Medical Expenditure Panel Survey (MEPS) [54] (medical visits). For all datasets we use the same protected groups as in [27]. Protected attributes are selected following standard fairness benchmarks.

Baselines. We compare MCGRAD against two multicalibration baselines: Discretization-Free MultiCalibration (DFMC) [30] and HKRR Hebert-Johnson et al. [28]. DFMC shares some similarities with MCGRAD: It fits a single GBDT with a fixed maximum depth of 2 for each weak learner. Protected groups are explicitly specified by the user, rather than learned like in MCGRAD. Protected groups are provided to the algorithm as binary features that encode group membership. HKRR is the boosting procedure proposed by Hebert-Johnson et al. [28], for which we use Hansen et al. [27]’s implementation³. We also include Logistic Regression as BASEPRED, the base predictor f_0 shared by all methods, and ISOTONIC [62] (Isotonic Regression), a widely adopted calibration algorithm.

Metrics. We evaluate MCGRAD relative to baselines on three dimensions: the effect on 1) *predictive performance*, 2) *calibration*, and 3) *multicalibration*.

1) *Predictive performance.* We employ the Area Under Precision-Recall Curve (PRAUC) [12] and the log loss.

2) *Calibration.* Various calibration metrics have been proposed in the literature. The *Expected Calibration Error (ECE)* [44] is the most commonly used calibration measure in machine learning, which bins the model scores and computes the deviation from perfect calibration, i.e., the absolute difference between the model’s predicted accuracy and its empirical accuracy [44]. Binning-based calibration metrics, including ECE, vary significantly based on the choice of bins. This is a well-documented shortcoming [1, 50]. Alternative metrics that aim to circumvent the drawbacks of binning include *smooth ECE (smECE)* which relies on kernel smoothing [4], but merely replaces sensitivity to the arbitrary choice of binning with sensitivity to an arbitrary choice of kernel [1]. The *Estimated Cumulative Calibration Error (ECCE)* [1] is a calibration metric that is rooted in cumulative statistics that does not require making an arbitrary choice in bin or kernel, which we covered in more detail in Section 2. *Brier score* is a proper scoring rule that is commonly used to evaluate probabilistic forecasts that is essentially just the mean squared error of predicted probability. Brier score decomposes into calibration and the model’s ability to separate the positive and negative class [5]. In this paper, we quantify calibration error using

ECCE, with exceptions in Section 5, where for pragmatic reasons we resort to ECE and Brier score whenever ECCE was not measured in the production system.

3) *Multicalibration.* Blasiok et al. [3] proposed the *maximum group-wise smECE* metric, which was used for empirical evaluations in Hansen et al. [27]. Guy et al. [25] proposed an extension of ECCE to quantify multicalibration, called *Multicalibration Error (MCE)*, which we defined in Eq. (2). *Maximum group-wise smECE* and *MCE* have in common that they both are defined as a maximum of some quantity over groups, but differ in the calibration quantity calculated in each group, where they respectively use smECE and ECCE. In this paper, we use MCE to quantify multicalibration. In the online supplement⁴, we additionally report group-wise smECE, thereby providing results that are in-line with [27]. We can measure any multicalibration metric with respect to either of two sets of groups. *Prespecified Groups*: refers to a small (less than 15 across all datasets) set of protected groups that are formed using various features of the datasets. For this, we use the groups specifications that were used in Hansen et al. [27]. *Unspecified Groups*: covers the case where the user does not specify which groups to protect against. In that case, the scores are expected to be multicalibrated with respect to all possible groups, and to calculate this, an extensive set of combinatorially generated groups are used.

Hyperparameters. MCGRAD uses LIGHTGBM as GBDT implementation. To ensure seamless adoption by ML engineers, we set default hyperparameters so that MCGRAD works out-of-the-box, without requiring application-specific tuning in the majority of use-cases. We determined the default hyperparameters by performing a grid search over 35 company-internal datasets and selecting the configuration that never degraded the base model’s log loss or PRAUC, and minimized the average MCE across datasets. The resulting hyperparameters are: `LEARNING_RATE= 0.02873`, `MAX_DEPTH= 5`, `MIN_CHILD_SAMPLES = 160`, `N_ESTIMATORS= 94`, `NUM_LEAVES= 5`, `LAMBDA_L2= 0.00913`, `MIN_GAIN_TO_SPLIT= 0.15`. All other parameters are set as for LIGHTGBM defaults.

For HKRR we follow Hansen et al. [27] and pick the best hyperparameters from a set of four specifications using a held-out validation set. For DFMC we use the default hyperparameters of LIGHTGBM since the paper does not suggest any specific hyperparameters.

³<https://github.com/dutchhansen/empirical-multicalibration>

⁴https://github.com/facebookresearch/mcgrad_multicalibration_at_web_scale

Baseline	MCE		log loss		PRAUC		ECCE	
	AVG	RANK	AVG	RANK	AVG	RANK	AVG	RANK
MCGRAD	6.60	1.18	0.375	1.00	0.714	1.00	1.97	2.27
Isotonic	24.00	2.91	0.410	3.27	0.662	3.91	1.53	1.54
DFMC	24.95	3.00	0.408	2.27	0.671	2.18	2.20	2.91
HKRR	25.74	3.73	0.414	4.00	0.637	5.00	3.29	3.54
BasePred	29.07	4.18	0.414	4.45	0.668	2.91	4.66	4.73

Table 2: Average value (AVG) and rank (RANK) over the datasets, for various metrics. MCGRAD has the best average log loss and PRAUC, and second-best average ECCE.

4.2 Experimental Results

Q1. MCGRAD vs existing baselines on *unspecified* groups.

Figure 1 (left-hand side) shows the Multicalibration Error (MCE) for all compared methods using all available features. Overall, MCGRAD outperforms all baselines on 10 out of the 11 datasets by reducing the base predictor’s MCE between 4% (Credit) and 93% (ACSEmploy), and obtaining an average reduction of 56.1%. In contrast, DFMC, ISOTONIC, and HKRR only achieve an average reduction of, respectively, 11.9%, 9.6%, and 1.4%. In addition, MCGRAD never harms the base predictor’s performance: even on MEPS, where there is low evidence of BASEPRED’s miscalibration (3.9 MCE), it slightly improves the MCE by 2.8%. On the other hand, ISOTONIC, DFMC, and HKRR increase (worsen) the MCE of BASEPRED on 3, 3 and 4 datasets respectively.

Table 2 reports the average performance over the datasets. MCGRAD achieves by far the best average log loss improvement (10.4%) and PRAUC improvement (8.1%) over the base predictor. Besides these average improvements over datasets, MCGRAD never harms performance, which helps build the necessary trust among ML engineers to deploy it in production environments. Other methods don’t have this guarantee. For instance, while DFMC improves model performance metrics by a lower 1.8% and 0.76%, it harms PRAUC and log loss on two datasets. ISOTONIC and HKRR even decrease the PRAUC on average by, respectively, 1% and 5% with a negligible change in log loss. For each metric, we rank all compared methods from 1 (best) to 5 (worst) and report the average ranking position in Table 2. A lower average rank indicates how often the method is preferred, as opposed to the average performance, which reflects the magnitude. MCGRAD consistently ranks as #1 baseline on log loss and PRAUC, while only ranking third for MCE on the MEPS dataset. Finally, MCGRAD ranks second best method in terms of global calibration (ECCE), losing against Isotonic Regression, which is expected because it is trained to directly optimize the global calibration of the base predictor.

Q2. MCGRAD vs existing baselines on *prespecified* groups.

To verify that good performance on the MCE metric for all groups implies also protection for prespecified groups, we report results just for these groups in Figure 1 (right-hand side). Overall, MCGRAD outperforms all baselines on 5 out of 11 datasets achieving a 54% average MCE improvement, and is followed by DFMC that is the best baseline on three datasets (ACSMOBILITY, HDMA, ACSTRavel) and achieves an average MCE improvement of 51%. HKRR obtains an average improvement in MCE of 36%, while ISOTONIC only achieves

an improvement of 16%. Note that HKRR and DFMC have direct access to these groups, while MCGRAD only has access to the features used to define the groups.

Q3. The impact of multiple rounds on MCGRAD. Because MCGRAD recurrently improves its own predictions by using T rounds, one relevant question is whether $T > 1$ is really necessary in practice. For this goal, we compare MCGRAD with its variant ONLY ONE ROUND that forces the number of rounds $T = 1$.⁵ Figure 2 (purple line) shows the relative improvement of ONLY ONE ROUND for the four considered metrics and all benchmark datasets. In the majority of cases, it is evident that ONLY ONE ROUND deteriorates MCGRAD’s performance: on the MCE and ECCE axes, it more than doubles its value (relative improvement < -100) for, respectively, 6 and 5 datasets, while, on the log loss and PRAUC axes, it worsens the performance by approximately 2% or more on, respectively, 8 and 6 out of 11 datasets. Interestingly, the largest deterioration of performance corresponds to MCGRAD running for the highest number of rounds: the optimal T for ACSEMPLOY, ACSINCOME, ACSPUBLIC, and ACSMOBILITY is, respectively, 20, 35, 27 and 22.

Q4. Impact of rescaling the logits and the regularization through *min sum Hessian in leaf*.

Rescaling the logits of all predictions by learning a constant speeds up MCGRAD training by reducing rounds, especially on very large datasets. As a downside, it might affect MCGRAD’s performance. Figure 2 (light blue) shows the relative deterioration after removing rescaling. Effects are mild and balanced: removal worsens MCE, log loss, and PRAUC on 6, 6, and 7 datasets respectively, with average drops of 9%, 0.5%, 0.03%.

LightGBM’s default *min sum Hessian leaf* (MSHL) is 0.001 with a min leaf sample size of 20. We claim that higher MHL values prevent probabilities from pushing to extremes, reducing overfitting (Section 3.3). We compare MCGRAD with MHL=0.001 versus our default of 20. Figure 2 (dark blue bar) shows mild deterioration using 0.001, likely due to limited dataset sizes. On average, lower MHL slightly harms performance, but this becomes evident on ACSEMPLOY (the largest dataset), where the MHL variant runs only 2 rounds (vs. 10) because our regularization mechanism stops the training earlier to avoid overfitting.

Q5. Computational time of MCGRAD vs multicalibration baselines as the number of groups scales.

Scalability can be a relevant bottleneck for multicalibration deployment in production environments. To evaluate this, we measure the training (fit) and inference (predict) times of MCGRAD, HKRR, and DFMC as the number of protected groups increases from 10 to 1000, using $\approx 1M$ training and $\approx 300k$ test samples from the ACSEmploy dataset.

Figure 3 shows the methods’ computational time per sample (in microseconds) as a function of the number of groups. Notably, the gap between MCGRAD and each baseline becomes larger as group count increases: at 1000 groups, MCGRAD maintains nearly constant inference time ($\sim 20\mu s$) while HKRR and DFMC require, respectively, 5019 and 1292 μs , meaning MCGRAD achieves less than 2% of their latency overhead. This gap becomes even larger in production settings where models with 100s of features can generate tens of thousands of potential protected groups.

⁵Early stopping can possibly return $T = 0$.

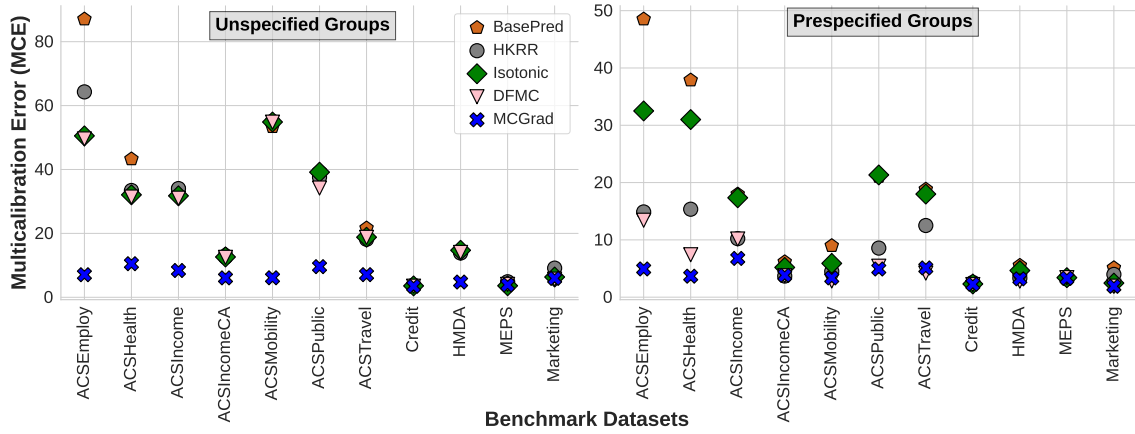


Figure 1: Multicalibration Error computed using *unspecified* groups (left) or the manually *prespecified* groups (right) for all compared methods on each benchmark dataset. Overall, MCGRAD achieves a better (lower) error for 10 out of 11 datasets when tested on unspecified groups, and for 5 out of 11 datasets when tested on prespecified groups.

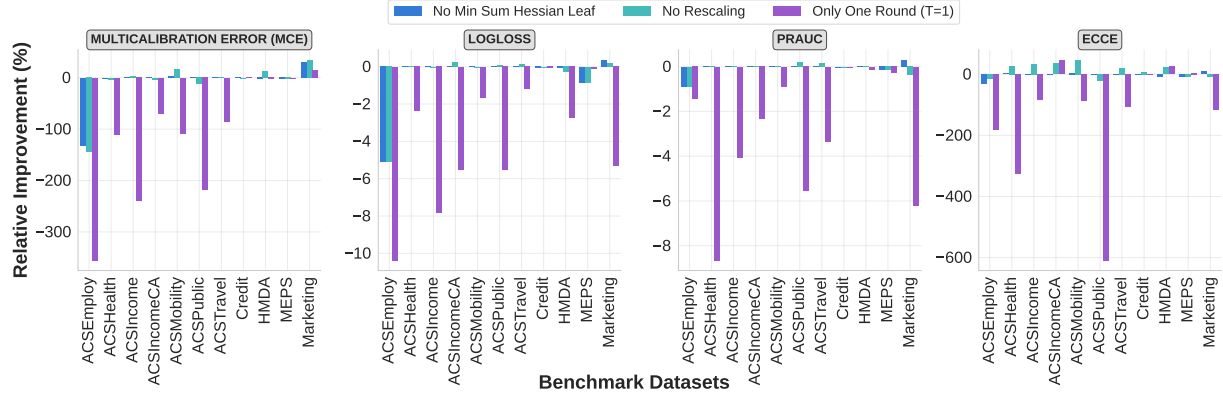


Figure 2: Improvement (in %) of MCGRAD’s variants relative to the original version. While setting $T = 1$ yields a significant drop in performance, the effect of the rescaling factor and min sum Hessian in leaf are mild due to the datasets limited size.

More importantly, these seemingly small latency differences have strong business impact. Production models typically serve online inference where even millisecond-scale increases in response time negatively affect the system performance and the user experience. Existing research claims that an *increase* of latency directly correlates with *reduced* user engagement and online business metrics [36]. MCGRAD’s sub-20 μ s inference overhead allows deployment in production, while baseline methods requiring few milliseconds per prediction can be impractical for production use.

5 Results in Production Deployment

We describe the results obtained by using MCGRAD in a real-world production system. MCGRAD has been deployed and used in production at Meta. Specifically, we have integrated MCGRAD into two ML training platforms at Meta: the publicly released LOOPER [42], and an internal platform that we refer to as MLPLATFORM#2. The evaluation consists of a total of 147 binary classification models that serve live production traffic: 27 models on Looper and 120

models on the other platform. On both ML training platforms, we run MCGRAD with default hyperparameters.

LOOPER Results. LOOPER [42] is a system that automates model training, online A/B test comparisons between the new model variant and the existing production model, and launch decisions on whether or not to promote the new model variant. These launch decisions are made based on whether the new model variant outperforms the existing production model on the selected evaluation metric with statistical significance.

For 27 binary classification models that serve active production traffic on LOOPER, we created a variant of the model that applies MCGRAD as calibration post-processing. We started online A/B tests to compare these 27 models against the active production models, all of which are GBDTs. Those production models all had previously applied Platt scaling [49].

We found that on 24 out of 27 models, the variant with MCGRAD statistically significantly outperformed the same model with Platt scaling [49], resulting in the promotion of the MCGRAD-calibrated model variant to become the primary production model.

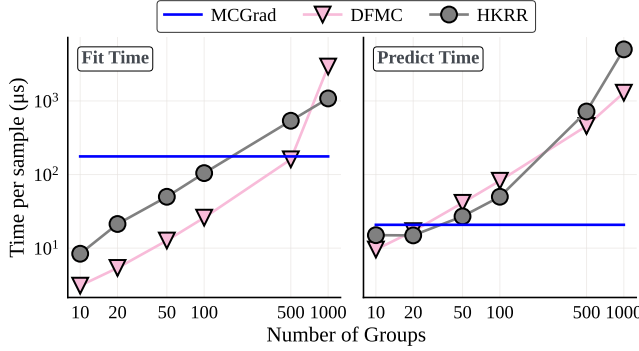


Figure 3: Computational time of multicalibration algorithms with varying numbers of groups. (Left) Fit time and (Right) predict time in seconds as a function of the number of groups on a log-log scale. Runtime for DFMC and HKRR increases with the number of groups, while MCGrad is constant.

Model	PRAUC ($\uparrow$)		Brier Score ($\downarrow$)	
	PLATT SCALING	MCGRAD	PLATT SCALING	MCGRAD
#1	0.5295	0.6565 (+23.9%)	Not available	Not available
#2	0.1824	0.2433 (+33.4%)	0.0117	0.0116 (-0.45%)
#3	0.2161	0.2300 (+6.4%)	0.0642	0.0632 (-1.56%)
#4	0.6268	0.6354 (+1.4%)	0.0394	0.0365 (-7.24%)

Table 3: Online results for four models on Looper, where either Platt Scaling or MCGRAD is used for post-processing.

Percentile	First Time Period				Second Time Period			
	Log loss	PRAUC	AUROC	ECE	Log loss	PRAUC	AUROC	ECE
10th	-26.50%	+3.58%	+3.15%	-93.88%	-16.34%	+6.31%	+1.97%	-90.84%
25th	-10.35%	+0.93%	+0.52%	-87.45%	-3.43%	+1.92%	+1.10%	-83.07%
50th	-1.39%	+0.32%	+0.17%	-53.83%	-0.86%	+0.06%	+0.16%	-58.19%
75th	-0.51%	+0.00%	+0.03%	-14.54%	-0.11%	-0.95%	0.0%	-21.96%
90th	+0.02%	-2.07%	-0.20%	+7.56%	+1.75%	-4.85%	-0.39%	+26.23%

Table 4: Summary statistics for the two time periods of the impact of MCGRAD on 120 (first period, left) and 65 (second period, right) ML models on MLPLATFORM#2.

Looper also monitors area under the *precision-recall curve* (PRAUC) on online production data. PRAUC improved on 24 of the 27 models and was neutral on the rest. Due to data retention, we are able to calculate the exact PRAUC for four of the 27 Looper models for which we most recently deployed MCGRAD, as well as the Brier score [6]. Table 3 summarizes the results on those models.

MLPLATFORM#2 Results. This ML training platform is widely used at Meta. ML model training scripts on this platform define the training set, a validation set, the test set, the ML model architecture, and the set of model evaluation metrics. The ML platform allows running an *evaluation flow* that trains the model on the training set and evaluates the model on the test set using the specified set of evaluation metrics, or a *publish flow* that prepares an API endpoint that can generate predictions for live production traffic.

We integrated MCGRAD into the evaluation flow so that we obtain an additional set of evaluation results for MCGRAD.

For every ML model evaluation on this ML platform, we collected parallel evaluation results *with* and *without* MCGRAD. We conducted data collection in two separate periods. Table 4 summarizes the results. In the first data collection period, we obtained results on 120 binary classification production models. We found that applying MCGRAD post-processing improved log loss for 88.7% of the models, compared to the production models without it. PRAUC improved for 76.7% of the models, AUROC for 80.2%, and Expected Calibration Error (ECE) [44] for 86%. Note that on most models where no improvement was found, there was no metric degradation either, due to the early stopping. In Table 4 we can see that the worst 10th-percentile metric effects of MCGRAD still result in degradations. We manually investigated a sample of those models, and found that these were mostly explained by errors by ML practitioners in the train and test set specification. E.g., in some cases the train and test set were clearly from a different population, or sample weights were defined in the train set but not in the test set.

In a second data collection period, we obtained results on 65 binary classification models. We found that MCGRAD provided improvements over the production models on the log loss metric for 80.3% of the models, on the PRAUC metric for 75.4% of the models, on the AUROC for 79.3%, and ECE for 86%.

6 Learnings from Production Deployment

Despite the recent focus of the academic community on multicalibration, it has seen little to no uptake in industry. Here we summarize various learnings from applying multicalibration in industry through production deployments of MCGRAD and of the Multicalibration Error metric (MCE) [25].

Learning 1. Multicalibration has business value.

Our results demonstrate that MCGRAD can improve real-world models’ multicalibration as well as performance metrics that tend to correlate with business outcomes, such as PRAUC. Furthermore, baselines such as Isotonic Regression do not achieve this. This contradicts observations by Hansen et al. [27] that (1) Isotonic Regression is often competitive compared to algorithms designed specifically for multicalibration, and (2) practitioners may face a trade-off between multicalibration and predictive performance.

Learning 2. Practitioners struggle to define protected groups.

Many multicalibration algorithms [28, 30] require manual specification of the protected groups. In our experience deploying multicalibration in production, we found that many ML engineers struggle to manually define relevant protected groups, and lack guidelines and frameworks for defining them. This experience is in line with findings from a series of interviews on AI fairness adoption in industry [29], where it turns out that practitioners often lack knowledge on different types of biases and cannot identify the relevant biases to correct for. Hence, our experience is that in order to succeed in obtaining industry adoption, a method needs to relieve users of the burden of manually defining protected groups, like MCGRAD does.

Learning 3. Practitioners consider adopting multicalibration methods only when they work *out-of-the-box*, reliably, and with no risk of harming model performance.

Many multicalibration algorithms have hyperparameters. We found industry practitioners to be open to applying multicalibration to production ML models, but only if it takes limited effort. Hyperparameter tuning is often seen as complex or time-consuming. They also perceive it as a reliability risk: if a method requires hyperparameter tuning to work well, then they wonder if the hyperparameter configuration will still work well in subsequent training runs.

Therefore, for a multicalibration algorithm to find successful adoption in industry, it is important that it works *out-of-the-box*, without the need for hyperparameter tuning. Moreover, while practitioners care about multicalibration, we found that they are not willing to accept a degradation in metrics like the log loss and the PRAUC, which are often believed to relate to topline metrics.

We defined a set of hyperparameter default values for MCGRAD that achieves (1) no degradation in PRAUC and log loss, and (2) substantial reduction in MCE. By pre-computing defaults through meta-analysis across 35 datasets, we enable MCGRAD to deliver consistent improvements without requiring domain expertise or extensive experimentation. Results in Sections 4 and 5 confirm that those default values consistently produce log loss and PRAUC improvements over models calibrated with Platt scaling (on Looper) or uncalibrated models (on MLPLATFORM#2), and consistently yield log loss, PRAUC, and MCE improvements on the public datasets.

7 Related Work

Multicalibration was introduced by Hebert-Johnson et al. [28] as a learning objective requiring predictors to be calibrated across a large collection of subgroups. Subsequent work has deepened its theoretical foundations, exploring computational complexity and connections to other learning paradigms [3, 13, 14, 21].

Extensions such as *low-degree multicalibration* [22] generalize subgroup definitions using weight functions, showing that restricting to low-degree polynomials can reduce computational complexity. *Swap multicalibration* [21] strengthens the original concept and is satisfied by some existing algorithms; it is also equivalent to *swap omniprediction* for certain loss function classes. The notion of *omnipredictors* [20] further generalizes agnostic learning by requiring minimization guarantees for a class of loss functions [46]. Connections to *multi-objective learning* have also been explored, framing multicalibration as a game dynamics problem [26].

Algorithms for multicalibration. The early proposals of multicalibration algorithms, such as HKRR [28] (e.g., implemented in MCBoost [48] or by Hansen et al. [27]) and LSBoost [19], rely on discretizing the output space and calibrating the prediction for each discretization level until convergence. A discretization-free multicalibration algorithm was proposed in [30], requiring group functions as input and using a gradient boosting machine to fit an ensemble of depth-two decision trees to the residuals of an uncalibrated predictor using both the original features and the predictions of the uncalibrated predictor. For guaranteeing multicalibration within an additive error, this algorithm requires a loss saturation condition to hold, i.e., that the squared-error loss can be

further reduced only by a small amount. Our algorithm uses gradient boosting similarly but does not require pre-specified groups. Instead, it leverages multi-round recursive gradient boosting and supports arbitrary user-defined decision trees in each round.

8 Conclusion & Limitations

We tackle the problem of designing a multicalibration algorithm without having access to pre-specified protected sets, that is fast and safe to deploy in production. These three aspects are relevant for practitioners as they are often (1) unable to define groups for their task, (2) forced to limit the overhead in terms of memory and speed, and (3) required to guarantee that the algorithm does not harm the base predictor. We proposed MCGRAD, a novel algorithm that achieves multicalibration by finding and fixing regions of the feature space where the miscalibration is large. Because this might create new regions with evidence of miscalibration, MCGRAD employs multiple rounds, where in each round it corrects its own output. In addition, MCGRAD’s implementation is fast, lightweight, and safe to deploy. We showed that MCGRAD outperforms existing baselines on benchmark datasets and provided results obtained in a large-scale industry production deployment at Meta consisting of hundreds of production models. Finally, we included practical takeaways from our experience applying multicalibration in industry.

Limitations. Although MCGRAD shows improvements, several limitations remain. First, guardrails against harming the base predictor may not be sufficient under concept drift. Specifically, MCGRAD can potentially worsen performance on post-drift data distributions, as multicalibration is only robust to certain types of distribution shift [34, 51]. Second, MCGRAD is currently limited to binary classification settings. Although multiclass extensions exist, they often suffer from reduced sample efficiency and increased complexity, creating challenges for real-world deployment [63]. Third, while MCGRAD can be applied to other data modalities such as text and images (using embeddings), these applications have not been explored in this paper. Future research may study MCGRAD’s (1) robustness to broader forms of drift, (2) extension to multiclass/regression problems, and (3) effectiveness on other data modalities.

Acknowledgments

We gratefully acknowledge Nicolas Stier, Udi Weinsberg, Ido Guy, Thomas Leeper, and Mark Tygert for their valuable discussions and support, which contributed to the success of this work.

References

- [1] Imanol Arrieta-Ibarra, Paman Gujral, Jonathan Tannen, Mark Tygert, and Cherie Xu. 2022. Metrics of calibration for probabilistic predictions. *Journal of Machine Learning Research* 23, 351 (2022), 1–54.
- [2] Antonio Bella, Cèsar Ferri, José Hernández-Orallo, and María José Ramírez-Quintana. 2010. Calibration of machine learning models. In *Handbook of Research on Machine Learning Applications and Trends: Algorithms, Methods, and Techniques*. IGI Global Scientific Publishing, 128–146.
- [3] Jarosław Blasiok, Parikshit Gopalan, Lunjia Hu, Adam Tauman Kalai, and Preetum Nakkiran. 2024. Loss Minimization Yields Multicalibration for Large Neural Networks. In *Innovations in Theoretical Computer Science Conference (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 287)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 17:1–17:21.
- [4] Jarosław Blasiok and Preetum Nakkiran. 2023. Smooth ECE: Principled reliability diagrams via kernel smoothing. arXiv:2309.12236 [cs.LG]

- [5] Gail Blattenberger and Frank Lad. 1985. Separating the Brier score into calibration and refinement components: A graphical exposition. *The American Statistician* 39, 1 (1985), 26–32.
- [6] Glen W Brier. 1950. Verification of forecasts expressed in terms of probability. *Monthly Weather Review* 78, 1 (1950), 1–3.
- [7] Sougata Chaudhuri, Abraham Bagherjeiran, and James Liu. 2017. Ranking and calibrating click-attributed purchases in performance display advertising. In *Proceedings of the ADKDD'17*. 1–6.
- [8] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) (KDD '16). Association for Computing Machinery, New York, NY, USA, 785–794.
- [9] Zhenbang Chen and Peng Qi. 2022. *Calibrating CTR Prediction with Transfer Learning in Instacart Ads*. Instacart Tech Blog. <https://tech.instacart.com/calibrating-ctr-prediction-with-transfer-learning-in-instacart-ads-3ec88fa97525>
- [10] A Feder Cooper, Solon Barocas, Christopher De Sa, and Siddhartha Sen. 2023. Variance, self-consistency, and arbitrariness in fair classification. arXiv:2301.11562 [cs.LG]
- [11] Diego Corrêa da Silva and Dietmar Jannach. 2025. Calibrated Recommendations: Survey and Future Directions. *arXiv preprint arXiv:2507.02643* (2025).
- [12] Jesse Davis and Mark Goadrich. 2006. The relationship between Precision-Recall and ROC curves. In *International Conference on Machine Learning*. 233–240.
- [13] Zhun Deng, Cynthia Dwork, and Linjun Zhang. 2023. HappyMap: A Generalized Multicalibration Method. In *Innovations in Theoretical Computer Science Conference (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 251)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 41:1–41:23.
- [14] Gianluca Detommaso, Martin Bertran Lopez, Riccardo Fogliato, and Aaron Roth. 2024. Multicalibration for Confidence Scoring in LLMs. In *International Conference on Machine Learning*.
- [15] Frances Ding, Moritz Hardt, John Miller, and Ludwig Schmidt. 2021. Retiring adult: New datasets for fair machine learning. *Advances in Neural Information Processing Systems* 34 (2021), 6478–6490.
- [16] Cynthia Dwork, Michael P Kim, Omer Reingold, Guy N Rothblum, and Gal Yona. 2022. Beyond bernoulli: Generating random outcomes that cannot be distinguished from nature. In *International Conference on Algorithmic Learning Theory*. PMLR, 342–380.
- [17] Yewen Fan, Nian Si, and Kun Zhang. 2023. Calibration Matters: Tackling Maximization Bias in Large-scale Advertising Recommendation Systems. In *International Conference on Learning Representations*.
- [18] Sarah Flood, Miriam King, Renae Rodgers, Steven Ruggles, J. Robert Warren, and Michael Westberry. 2021. Integrated Public Use Microdata Series, Current Population Survey: Version 9.0. IPUMS. <https://cps.ipums.org/cps/>
- [19] Ira Globus-Harris, Declan Harrison, Michael Kearns, Aaron Roth, and Jessica Sorrell. 2023. Multicalibration as Boosting for Regression. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 11459–11492.
- [20] Parikshit Gopalan, Adam Tauman Kalai, Omer Reingold, Vatsal Sharan, and Udi Wieder. 2022. Ominipredictors. In *Innovations in Theoretical Computer Science Conference (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 215)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 79:1–79:21.
- [21] Parikshit Gopalan, Michael Kim, and Omer Reingold. 2023. Swap Agnostic Learning, or Characterizing Ominiprediction via Multicalibration. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 39936–39956.
- [22] Parikshit Gopalan, Michael P. Kim, M. Singhal, and Shengjia Zhao. 2022. Low-Degree Multicalibration. In *Conference on Learning Theory*.
- [23] Mert Gulsoy, Emre Yalcin, Yucel Tacli, and Alper Bilge. 2025. DUOR: Dynamic User-oriented re-Ranking calibration strategy for popularity bias treatment of recommendation algorithms. *International Journal of Human-Computer Studies* (2025), 103578.
- [24] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. 2017. On calibration of modern neural networks. In *International Conference on Machine Learning*. PMLR, 1321–1330.
- [25] Ido Guy, Daniel Haimovich, Fridolin Linder, Nastaran Okati, Lorenzo Perini, Niek Tax, and Mark Tygert. 2025. Measuring multi-calibration. arXiv:2506.11251 [stat.ME] <https://arxiv.org/abs/2506.11251>
- [26] Nika Haghtalab, Michael Jordan, and Eric Zhao. 2023. A Unifying Perspective on Multi-Calibration: Game Dynamics for Multi-Objective Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 72464–72506. https://proceedings.neurips.cc/paper_files/paper/2023/file/e55edcd01ac45c839a602f9e09f9cb-Paper-Conference.pdf
- [27] Dutch Hansen, Siddhartha Devic, Preetum Nakkiran, and Vatsal Sharan. 2024. When is multicalibration post-processing necessary? *Advances in Neural Information Processing Systems* 37 (2024), 38383–38455.
- [28] Ursula Hebert-Johnson, Michael Kim, Omer Reingold, and Guy Rothblum. 2018. Multicalibration: Calibration for the (Computationally-Identifiable) Masses. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, 1939–1948.
- [29] Kenneth Holstein, Jennifer Wortman Vaughan, Hal Daumé III, Miro Dudik, and Hanna Wallach. 2019. Improving fairness in machine learning systems: What do industry practitioners need?. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–16.
- [30] Hongyi Henry Jin, Zijun Ding, Dung Daniel Ngo, and Zhiwei Steven Wu. 2025. Discretization-free Multicalibration through Loss Minimization over Tree Ensembles. arXiv:2505.17435 [cs.LG] <https://arxiv.org/abs/2505.17435>
- [31] Christopher Jung, Changhwa Lee, Mallesh Pai, Aaron Roth, and Rakesh Vohra. 2021. Moment multicalibration for uncertainty estimation. In *Conference on Learning Theory*. PMLR, 2634–2678.
- [32] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 3149–3157.
- [33] Alex Kendall and Yarin Gal. 2017. What uncertainties do we need in bayesian deep learning for computer vision? *Advances in Neural Information Processing Systems* 30 (2017).
- [34] Michael P Kim, Christoph Kern, Shafi Goldwasser, Frauke Kreuter, and Omer Reingold. 2022. Universal adaptability: Target-independent inference that competes with propensity scoring. *Proceedings of the National Academy of Sciences* 119, 4 (2022), e2108097119.
- [35] Ian D. Kivlichan, Zi Lin, Jeremiah Liu, and Lucy Vasserman. 2021. Measuring and Improving Model-Moderator Collaboration using Uncertainty Estimation. In *WOAH 2021 - 5th Workshop on Online Abuse and Harms, Proceedings of the Workshop*.
- [36] Ron Kohavi, Alex Deng, Roger Longbotham, and Ya Xu. 2014. Seven rules of thumb for web site experimenters. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1857–1866.
- [37] Meelis Kull, Miquel Perello Nieto, Markus Kängsepp, Telmo Silva Filho, Hao Song, and Peter Flach. 2019. Beyond temperature scaling: Obtaining well-calibrated multi-class probabilities with dirichlet calibration. *Advances in Neural Information Processing Systems* 32 (2019).
- [38] Meelis Kull, Telmo Silva Filho, and Peter Flach. 2017. Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 623–631.
- [39] Wonbin Kweon, SeongKu Kang, and Hwanjo Yu. 2022. Obtaining calibrated probabilities with personalized ranking models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 4083–4091.
- [40] Hongfu Liu, Hengguan Huang, Xiangming Gu, Hao Wang, and Ye Wang. 2024. On calibration of LLM-based guard models for reliable content moderation. *arXiv preprint arXiv:2410.10414* (2024).
- [41] Haihao Lu and Rahul Mazumder. 2020. Randomized Gradient Boosting Machine. *SIAM Journal on Optimization* 30, 4 (2020), 2780–2808.
- [42] Igor L Markov, Hanson Wang, Nitya S Kasturi, Shaun Singh, Mia R Garrard, Yin Huang, Sze Wai Celeste Yuen, Sarah Tran, Zehui Wang, Igor Glotov, et al. 2022. Looper: An end-to-end ml platform for product decisions. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3513–3523.
- [43] S. Moro, P. Rita, and P. Cortez. 2014. Bank Marketing. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5K306>.
- [44] Mahdi Pakdaman Naeini, Gregory Cooper, and Niles Hauskrecht. 2015. Obtaining well calibrated probabilities using bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 29.
- [45] Georgy Noarov and Aaron Roth. 2023. The statistical scope of multicalibration. In *International Conference on Machine Learning*. PMLR, 26283–26310.
- [46] Princewill Okoroafor, Robert Kleinberg, and Michael P. Kim. 2025. Near-Optimal Algorithms for Ominiprediction. arXiv:2501.17205 [stat.ML] <https://arxiv.org/abs/2501.17205>
- [47] Gustavo Penha and Claudia Hauff. 2021. On the calibration and uncertainty of neural learning to rank models. *arXiv preprint arXiv:2101.04356* (2021).
- [48] Florian Pfisterer, Christoph Kern, Susanne Dandl, Matthew Sun, Michael P. Kim, and Bernd Bischl. 2021. mcboost: Multi-Calibration Boosting for R. *Journal of Open Source Software* 6, 64 (aug 2021), 3453.
- [49] John Platt. 1999. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*. MIT Press, Cambridge, MA, 61–74.
- [50] Rebecca Roelofs, Nicholas Cain, Jonathon Shlens, and Michael C Mozer. 2022. Mitigating bias in calibration error estimation. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 4036–4054.
- [51] Aaron Roth. 2022. Uncertain: Modern topics in uncertainty quantification. <https://www.cis.upenn.edu/~aaroht/uncertainty-notes.pdf>
- [52] Eliran Shabat, Lee Cohen, and Yishay Mansour. 2020. Sample complexity of uniform convergence for multicalibration. In *Advances in Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 1118, 10 pages.

- [53] Shai Shalev-Shwartz and Shai Ben-David. 2014. *Understanding machine learning: From theory to algorithms*. Cambridge university press.
- [54] Shubham Sharma, Alan H Gee, David Paydarfar, and Joydeep Ghosh. 2021. Fair-n: Fair and robust neural networks for structured data. In *Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society*. 946–955.
- [55] Telmo Silva Filho, Hao Song, Miquel Perello-Nieto, Raul Santos-Rodriguez, Meelis Kull, and Peter Flach. 2023. Classifier calibration: a survey on how to assess and improve predicted class probabilities. *Machine Learning* 112, 9 (2023), 3211–3260.
- [56] Harald Steck. 2018. Calibrated recommendations. In *Proceedings of the 12th ACM conference on recommender systems*. 154–162.
- [57] Mark Tygert. 2023. Calibration of P-values for calibration and for deviation of a subpopulation from the full population. *Advances in Computational Mathematics* 49, 5 (2023), 70.
- [58] Bertie Vidgen, Scott A Hale, Sam Staton, Tom Melham, Helen Margetts, Ohad Kammar, and Marcin Szymczak. 2020. Recalibrating classifiers for interpretable abusive content detection. In *Proceedings of the Fourth Workshop on Natural Language Processing and Computational Social Science*. 132–138.
- [59] Jiayun Wu, Jiahuo Liu, Peng Cui, and Steven Z Wu. 2024. Bridging multicalibration and out-of-distribution generalization beyond covariate shift. *Advances in Neural Information Processing Systems* 37 (2024), 73036–73078.
- [60] Shuai Yang, Hao Yang, Zhuang Zou, Linhe Xu, Shuo Yuan, and Yifan Zeng. 2024. Deep Ensemble Shape Calibration: Multi-Field Post-hoc Calibration in Online Advertising. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6117–6126.
- [61] I-Cheng Yeh. 2009. Default of Credit Card Clients. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5S53H>.
- [62] Bianca Zadrozny and Charles Elkan. 2002. Transforming classifier scores into accurate multiclass probability estimates. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 694–699.
- [63] Shengjia Zhao, Michael Kim, Roshni Sahoo, Tengyu Ma, and Stefano Ermon. 2021. Calibrating predictions to decisions: A novel approach to multi-class calibration. *Advances in Neural Information Processing Systems* 34 (2021), 22313–22324.

A Bridging Multicalibration Error and α -MC

Several definitions of approximate multicalibration, known as α -multicalibration, exist in the literature. A common approach uses a uniform bound on absolute deviation $\Delta_{h,g}(f) \leq \alpha$ where the precise choice of h and g varies [3, 13, 22, 26]. Another common choice is a uniform bound on the conditional deviation [28, 52], i.e.

$$\delta_{h,g}(f) := |\mathbb{E}[h(X)g(f(X))(Y - f(X)) \mid h(X)g(f(X)) = 1]| \leq \alpha.$$

Recent work [26] improved these by showing a tighter bound $\Delta_{h,g}(f) \leq \alpha\sqrt{\mathbb{E}[h(X)]}$ can be achieved with no extra requirements.

We next show that Definition 2.1, for the expected deviation corresponding to the sample mean deviation over dataset D , i.e.,

$$\mathbb{E}_D[h(X)g(f(X))(Y - f(X))] = \frac{1}{n} \sum_{k=1}^n h(x_k)g(f(x_k))(y_k - f(x_k)),$$

with scale parameter chosen $\tau_h(f) = \sqrt{\mathbb{E}_D[h(X)f(X)(1 - f(X))]}$, matches with the MCE metric defined in Eq. (2).

PROPOSITION A.1. *Given a set $\mathcal{H}$ of group membership functions, a probabilistic predictor f is α -multicalibrated with respect to $\mathcal{H}$, with the scale parameter $\tau_h(f)$, if and only if $\text{MCE}(f) \leq \alpha\sqrt{n}$.*

PROOF. Note that $\Delta_{h,g}(f) = \frac{1}{n} |\sum_{k=1}^n h(x_k)g(f(x_k))(y_k - f(x_k))|$. Combining with the definition of $\text{ECCE}_h(f)$, we get that

$$\max_{g \in \mathcal{G}} \Delta_{h,g}(f) = \frac{n_h}{n} \text{ECCE}_h(f).$$

Plugging in the definition of $\text{MCE}(f)$, as in Eq. (2), we have

$$\frac{1}{\sqrt{n}} \text{MCE}(f) = \frac{1}{\sqrt{n}} \max_{h \in \mathcal{H}} \frac{\text{ECCE}_h(f)}{\sigma_h(f)} = \max_{h \in \mathcal{H}, g \in \mathcal{G}} \frac{\sqrt{n} \Delta_{h,g}(f)}{n_h \sigma_h(f)} = \max_{h \in \mathcal{H}, g \in \mathcal{G}} \frac{\Delta_{h,g}(f)}{\tau_h(f)}.$$

Now, from the last expression, α -MC with respect to $\mathcal{H}$ holds if and only if $\text{MCE}(f) \leq \alpha\sqrt{n}$. $\square$

B Convergence Analysis

We analyze the convergence of Algorithm 1 under specific conditions for the GBM used in each round. We show that (1) the loss does not increase with more rounds and decreases unless the predictor has converged, and (2) the α -multicalibration error after T rounds is bounded by the difference between predictors F_{T+1} and F_T and that 0-multicalibration is achieved at convergence. Unlike Section 3.1, which assumes adding an optimal linear combination of weak learners each round, here the GBM incrementally adds regression trees to F_t to form F_{t+1} .

We consider a GBM that, in round t , iteratively combines M_t regression trees as $F_t^{m+1}(x) = F_t^m(x) + \rho_m h_{j_m}(x, f_t(x))$ where j_m is the index of the regression tree added in the m -th iteration, ρ_m is the step size, and $m = 0, 1, \dots, M_t - 1$. Let

$$\mathcal{L}_t(w) = \mathbb{E}_D \left[\mathcal{L} \left(F_t(X) + \sum_{k=1}^K w_k h_k(X, f_t(X)), Y \right) \right].$$

The gradient of the loss function with respect to w_k is given by

$$\nabla_k \mathcal{L}_t(w) = \mathbb{E}_D [h_k(X, f_t(X))(Y - f_{t+1}(X))]$$

where $f_{t+1}(x) = \mu(F_t(x) + \sum_{k=1}^K w_k h_k(x, f_t(x)))$.

A GBM can be shown to correspond to a coordinate gradient descent in the coefficient space [41], with updates of the form $w^{m+1} = w^m - \rho \nabla_{j_m} \mathcal{L}_t(w^m) e_{j_m}$, where e_j is the j -th standard basis vector in $\mathbb{R}^K$ and ρ is the step size.

Decreasing Loss Property.

PROPOSITION B.1. *Assume that the loss function $\mathcal{L}$ is a convex and L -smooth (for the log loss, $L = 1/4$) and that the step size ρ is set as $\rho = 1/L$. Then, for every $t \geq 0$, the following inequality holds*

$$\mathbb{E}_D [\mathcal{L}(F_{t+1}(X), Y)] \leq \mathbb{E}_D [\mathcal{L}(F_t(X), Y)]$$

and is strict if $\nabla_{j_m} \mathcal{L}_t(w^m) \neq 0$ for some $m \in \{0, \dots, M_t - 1\}$.

PROOF. Note the following relations, for any ρ_m :

$$\mathcal{L}_t(w^{m+1}) \leq \mathcal{L}_t(w^m + \rho_m e_{j_m}) \leq \mathcal{L}_t(w^m) + \rho_m \nabla_{j_m} \mathcal{L}_t(w^m)^\top e_{j_m} + \frac{L}{2} \rho_m^2.$$

By taking $\rho_m = -(1/L) \nabla_{j_m} \mathcal{L}_t(w^m)$, we have

$$\mathcal{L}_t(w^{m+1}) \leq \mathcal{L}_t(w^m) - \frac{1}{2L} (\nabla_{j_m} \mathcal{L}_t(w^m))^2.$$

It follows that $\mathcal{L}_t(w^{M_t}) - \mathcal{L}_t(w^0) \leq -\frac{1}{2L} \sum_{m=0}^{M_t-1} (\nabla_{j_m} \mathcal{L}_t(w^m))^2$, which allows us to conclude that $\mathcal{L}_t(w^{M_t}) - \mathcal{L}_t(w^0) \leq 0$ where the inequality is strict if $\nabla_{j_m} \mathcal{L}_t(w^m) \neq 0$ for some $0 \leq m < M_t$. $\square$

For a GBM that, in each iteration, greedily selects the weak learner with the largest absolute value of the loss gradient, i.e., $j_m \in \arg \max_j |\nabla_j \mathcal{L}_t(w^m)|$, we can establish a convergence rate to the loss of the optimum linear combination of weak learners using Theorem 4.2 [41]. Let $w^*(t) = \arg \min_w \mathcal{L}_t(w)$. Then,

$$\mathbb{E}_D [\mathcal{L}(F_{t+1}(X), Y)] - \mathcal{L}_t(w^*(t)) \leq 0.5 \cdot \text{Dist}_t^2 \frac{1}{M_t} \quad (4)$$

where Dist_t denotes the distance between the level set $\{w : \mathcal{L}_t(w) \leq \mathcal{L}_t(w^0)\}$ and $w^*(t)$, as defined in [41].

This result justifies modeling each round as updating the predictor with an optimal ensemble of weak learners that minimizes loss.

Proposition B.1 shows the training loss never increases with more rounds and converges to a limit as rounds go to infinity.

A Bound on the Multicalibration Error. The bound in Proposition B.2 provides a theoretical upper-bound guarantee, showing that the multicalibration error converges to zero when C_T is bounded by a constant, the gap between prediction values F_{T+1} and F_T vanishes as the number of rounds T increases, and the GBM achieves optimal loss in round T . Before proving this, we need a preliminary result:

LEMMA B.1. *If f_T is such that $|\mathbb{E}[h(X)g(f_T(X))(y - f_T(X))]| \geq \alpha\sqrt{\mathbb{E}[h(X)f_T(X)(1 - f_T(X))]}$, for some $h \in \mathcal{H}$ and $g \in \mathcal{G}$, then there exists $\tilde{w} \in \mathbb{R}^K$ such that for $\tilde{h}(x) = \sum_{k=1}^K \tilde{w}_k h_k(x, f_T(x))$:*

$$\alpha \leq \frac{2}{\sqrt{3}} C_T \sqrt{\mathbb{E}[\mathcal{L}(F_T(X), Y)] - \mathbb{E}[\mathcal{L}(F_T(X) + \tilde{h}(X), Y)]}$$

$$\text{where } C_T^2 := \max_{h \in \mathcal{H}, g \in \mathcal{G}} \frac{\mathbb{E}[g(f_T(X)) | h(X) = 1]}{\mathbb{E}[f_T(X)(1 - f_T(X)) | h(X) = 1]}.$$

PROOF. For any F, h and any convex, L -smooth loss function $\mathcal{L}$,

$$\begin{aligned} \mathcal{L}(F(x), y) - \mathcal{L}(F(x) + h(x), y) &\geq \frac{d}{dF} \mathcal{L}(F(x) + h(x), y)(-h(x)) \\ &\geq \frac{d}{dF} \mathcal{L}(F(x), y)(-h(x)) - Lh(x)^2 \end{aligned}$$

where the first inequality holds as $\mathcal{L}$ is convex and the second inequality holds because $\mathcal{L}$ is L -smooth.

For the log loss $(d/dF)\mathcal{L}(F, y) = -(y - \mu(F))$, hence, we have

$$\mathbb{E}[\mathcal{L}(F(X), Y) - \mathcal{L}(F(X) + h(X), Y)] \geq \mathbb{E}[(Y - f_T(X))h(X)] - L\mathbb{E}[h(X)^2]$$

Let

$$(h^*, g^*) \in \arg \max_{h \in \mathcal{H}, g \in \mathcal{G}} \frac{\mathbb{E}[h(X)g(f_T(X))(Y - f_T(X))]}{\sqrt{\mathbb{E}[h(X)f_T(X)(1 - f_T(X))]}}$$

and, let $\tilde{w}$ be defined as:

$$\tilde{w}_{h,g} = \frac{\mathbb{E}[h(X)g(f_T(X))(Y - f_T(X))]}{\mathbb{E}[(h(X)g(f_T(X)))^2]}$$

if $(h, g) = (h^*, g^*)$, otherwise $\tilde{w}_{h,g} = 0$. For $w = \tilde{w}$, we have both

$$\begin{aligned} \mathbb{E}\left[(Y - f_T(X)) \sum_{k=1}^K w_k h_k(x, f_T(X))\right] &= \frac{\mathbb{E}[h^*(X)g^*(f_T(X))(Y - f_T(X))]^2}{\mathbb{E}[(h^*(X)g^*(f_T(X)))^2]} \\ \mathbb{E}\left[\left(\sum_{k=1}^K w_k h_k(X, f_T(X))\right)^2\right] &= \frac{\mathbb{E}[h^*(X)g^*(f_T(X))(Y - f_T(X))]^2}{\mathbb{E}[(h^*(X)g^*(f_T(X)))^2]}. \end{aligned}$$

Hence, we have

$$\begin{aligned} &\mathbb{E}\left[\mathcal{L}(F_T(X), Y) - \mathcal{L}\left(F_T(X) + \sum_{k=1}^K \tilde{w}_k h_k(X, f_T(X)), Y\right)\right] \\ &\geq (1 - L) \frac{\mathbb{E}[h^*(X)g^*(f_T(X))(Y - f_T(X))]^2}{\mathbb{E}[(h^*(X)g^*(f_T(X)))^2]} \\ &\geq (1 - L) \frac{\mathbb{E}[f_T(X)(1 - f_T(X)) | h^*(X) = 1]}{\mathbb{E}[g^*(f_T(X)) | h^*(X) = 1]} \alpha^2. \end{aligned}$$

Because $L = 1/4$ for the log loss, we conclude the proof

$$\alpha^2 \leq \frac{4}{3} C_T^2 \left(\mathbb{E}[\mathcal{L}(F_T(X), Y)] - \mathbb{E}\left[\mathcal{L}\left(F_T(X) + \tilde{h}(X), Y\right)\right] \right)$$

□

Now, we can derive the upper bound of the MC error:

PROPOSITION B.2. f_T is α -MC with $\alpha \leq \frac{2}{\sqrt{3}} C_T \sqrt{\phi_T + \epsilon_T}$ where

$$\begin{aligned} \phi_T &:= 2 \mathbb{E}[|F_{T+1}(X) - F_T(X)|] + \frac{1}{8} \mathbb{E}[(F_{T+1}(X) - F_T(X))^2] \\ \epsilon_T &:= \mathbb{E}[\mathcal{L}(F_{T+1}(X), Y)] - \mathcal{L}(w^*(T)), \end{aligned}$$

and C_T is defined as in Lemma B.1.

PROOF. For any $w \in \mathbb{R}^K$,

$$\begin{aligned} &\mathbb{E}[\mathcal{L}(F_t(X), Y)] - \mathbb{E}\left[\mathcal{L}\left(F_t(X) + \sum_{k=1}^K w_k h_k(X, f_t(X)), Y\right)\right] \\ &\leq \underbrace{\mathbb{E}[\mathcal{L}(F_t(X), Y) - \mathcal{L}(F_{t+1}(X), Y)]}_{(*)} + \underbrace{\mathbb{E}[\mathcal{L}(F_{t+1}(X), Y)] - \mathcal{L}(w^*(t))}_{(**)} \end{aligned}$$

The first term $(*)$ captures the gap of the loss function values at two successive rounds of the algorithm and can be bounded by $2\mathbb{E}[|F_{t+1}(X) - F_t(X)|] + \frac{L}{2}\mathbb{E}[(F_{t+1}(X) - F_t(X))^2]$. The second term $(**)$ measures the gap between the loss at the end of round t and the loss from an optimal linear combination of weak classifiers. This gap can be reduced by increasing the number of GBM iterations and is bounded as in Eq (4). The lemma follows from Lemma B.1. □

The bound on the multicalibration error depends on three terms: (a) C_T , which depends inversely on the average prediction variance within subgroups, (b) ϕ_T , which depends on the gap between the prediction values F_{T+1} and F_T , and (c) ϵ_T , which measures the gap between the loss achieved by the predictor F_{T+1} and the optimum loss in round T . If C_T is bounded by a constant, the MC error can be made arbitrarily small by reducing ϕ_T and ϵ_T . As T increases, ϕ_T approaches zero, and ϵ_T can be minimized by increasing the number of GBM iterations M_T , as shown in Eq. (4). Also, note that

$$C_T^2 \leq \frac{1}{\min_{h \in \mathcal{H}} \mathbb{E}[f_T(X)(1 - f_T(X)) | h(X) = 1]}.$$

This bound is finite if and only if $\mathbb{E}[f_T(X)(1 - f_T(X)) | h(X) = 1] > 0$ for every group $h \in \mathcal{H}$. Equivalently, for every group h , there exists at least one member x such that $0 < f_T(x) < 1$. This condition clearly holds if $0 < f_T(x) < 1$ for all $x \in \mathcal{X}$ which is satisfied whenever $F_T(x) \in [-R, R]$ for every $x \in \mathcal{X}$, for some constant $R > 0$. Moreover, if the predictor f_T takes values in $[\epsilon, 1 - \epsilon]$, for some $0 < \epsilon \leq 1/2$, then $C_T^2 \leq \frac{2}{\epsilon}$.

C Extended Experimental Results

dataset	mcb_algorithm	MCE * σ (unspecified)	MCE (unspecified)	MCE * σ (prespecified)	MCE (prespecified)	max smECE (prespecified)
MEPS	base_model	0.0265	3.9940	0.0926	3.2849	0.0819
	MCGrad_msh_20	0.0253	3.8815	0.0815	3.2923	0.0755
	MCGrad	0.0265	3.9940	0.0926	3.2849	0.0819
	MCGrad_group_features	0.0265	3.9940	0.0926	3.2849	0.0819
	MCGrad_no_unshrink	0.0265	3.9940	0.0926	3.2849	0.0819
	MCGrad_one_round	0.0256	3.9250	0.0806	3.2888	0.0733
	DFMC	0.0266	4.0473	0.0870	3.4147	0.0761
	HKRR	0.0318	4.8624	0.0872	3.1814	0.1054
	Isotonic	0.0237	3.6076	0.0797	3.4070	0.0713
acs_employment_all_states	base_model	0.0274	87.2951	0.0263	45.5082	0.0499
	MCGrad_msh_20	0.0019	7.1680	0.0099	4.7747	0.0137
	MCGrad	0.0085	30.7752	0.0126	14.2049	0.0199
	MCGrad_group_features	0.0143	47.9972	0.0061	3.3573	0.0120
	MCGrad_no_unshrink	0.0085	31.1755	0.0128	14.3882	0.0201
	MCGrad_one_round	0.0085	30.7752	0.0126	14.2049	0.0199
	DFMC	0.0156	52.0332	0.0110	12.6897	0.0152
	HKRR	0.0196	63.8249	0.0104	14.3973	0.0110
	Isotonic	0.0153	50.7122	0.0210	30.8645	0.0215
HMDA	base_model	0.0283	13.9421	0.0475	5.5320	0.0468
	MCGrad_msh_20	0.0091	4.7703	0.0300	3.2015	0.0365
	MCGrad	0.0093	4.8958	0.0309	3.2932	0.0371
	MCGrad_group_features	0.0283	13.9249	0.0372	2.8149	0.0363
	MCGrad_no_unshrink	0.0080	4.1592	0.0325	3.0728	0.0405
	MCGrad_one_round	0.0096	4.8650	0.0443	3.0720	0.0504
	DFMC	0.0287	14.0359	0.0372	2.8040	0.0380
	HKRR	0.0285	13.8664	0.0401	3.3343	0.0380
	Isotonic	0.0301	14.7130	0.0468	4.6516	0.0456
acs_travel_time_all_states	base_model	0.0192	21.5562	0.1736	18.6954	0.1688
	MCGrad_msh_20	0.0059	6.8154	0.0330	4.9704	0.0346
	MCGrad	0.0059	6.8237	0.0331	4.9730	0.0346
	MCGrad_group_features	0.0165	18.6758	0.0297	2.7626	0.0293
	MCGrad_no_unshrink	0.0061	6.9743	0.0349	4.1746	0.0365
	MCGrad_one_round	0.0118	13.3776	0.0865	12.9763	0.0840
	DFMC	0.0166	18.7304	0.0331	4.2345	0.0323
	HKRR	0.0164	18.4043	0.0355	12.7941	0.0349
	Isotonic	0.0167	18.7715	0.2116	18.1106	0.2025
acs_income_all_states	base_model	0.0233	34.0505	0.0714	17.9622	0.0708
	MCGrad_msh_20	0.0049	7.9316	0.0318	6.2694	0.0314
	MCGrad	0.0050	8.1133	0.0322	6.5280	0.0317
	MCGrad_group_features	0.0201	29.6245	0.0106	5.3609	0.0152
	MCGrad_no_unshrink	0.0049	7.8965	0.0317	4.5370	0.0305
	MCGrad_one_round	0.0179	27.7918	0.0520	13.9878	0.0502
	DFMC	0.0211	31.0460	0.0185	9.9542	0.0204
	HKRR	0.0230	33.8068	0.0408	10.2374	0.0413
	Isotonic	0.0216	31.7564	0.0701	17.3586	0.0694
ACSIIncome	base_model	0.0265	13.4846	0.0611	6.1578	0.0579
	MCGrad_msh_20	0.0104	6.0717	0.0443	3.8099	0.0498
	MCGrad	0.0102	5.9816	0.0439	3.8675	0.0501
	MCGrad_group_features	0.0239	12.3216	0.0320	3.4824	0.0337
	MCGrad_no_unshrink	0.0109	6.2974	0.0446	3.7393	0.0530
	MCGrad_one_round	0.0187	10.3174	0.0489	5.6888	0.0520
	DFMC	0.0243	12.5025	0.0345	3.7487	0.0351
	HKRR	0.0246	12.5983	0.0338	3.6878	0.0354
	Isotonic	0.0245	12.6166	0.0592	5.1895	0.0574
acs_health_insurance_all_states	base_model	0.0198	45.2690	0.0803	38.2892	0.0800
	MCGrad_msh_20	0.0044	10.4370	0.0082	4.0131	0.0108
	MCGrad	0.0044	10.4370	0.0082	4.0131	0.0108
	MCGrad_group_features	0.0120	27.8627	0.0038	3.2136	0.0075
	MCGrad_no_unshrink	0.0045	10.5774	0.0093	3.8013	0.0104
	MCGrad_one_round	0.0096	22.4316	0.0522	21.3152	0.0520
	DFMC	0.0118	27.4016	0.0148	7.5026	0.0232
	HKRR	0.0150	33.9543	0.0322	17.3882	0.0319
	Isotonic	0.0134	30.9506	0.0780	31.8857	0.0778
acs_public_health_insurance_all_states	base_model	0.0354	40.4553	0.1035	22.1733	0.1025
	MCGrad_msh_20	0.0068	8.3577	0.0324	4.9253	0.0400
	MCGrad	0.0067	8.2452	0.0320	4.9934	0.0396
	MCGrad_group_features	0.0282	32.7613	0.0305	3.4484	0.0324
	MCGrad_no_unshrink	0.0076	9.2211	0.0289	5.3953	0.0380
	MCGrad_one_round	0.0265	31.3918	0.0668	12.7760	0.0654
	DFMC	0.0277	32.0694	0.0411	6.4715	0.0434
	HKRR	0.0342	39.5956	0.0428	7.3886	0.0403
	Isotonic	0.0329	38.0286	0.1064	21.8710	0.1051
acs_mobility_all_states	base_model	0.0642	52.3930	0.0689	9.0919	0.0652
	MCGrad_msh_20	0.0068	5.8904	0.0306	3.1756	0.0349
	MCGrad	0.0068	5.8824	0.0302	3.2353	0.0347
	MCGrad_group_features	0.0673	55.1968	0.0156	2.4286	0.0200
	MCGrad_no_unshrink	0.0060	5.2134	0.0338	2.9911	0.0370
	MCGrad_one_round	0.0150	12.7685	0.0423	6.9712	0.0414
	DFMC	0.0671	54.9004	0.0217	2.9196	0.0261
	HKRR	0.0670	54.5418	0.0406	5.2062	0.0433
	Isotonic	0.0669	54.7596	0.0659	5.2414	0.0677
CreditDefault	base_model	0.0168	3.5652	0.0783	2.1844	0.0536
	MCGrad_msh_20	0.0162	3.4353	0.0842	2.2523	0.0543
	MCGrad	0.0162	3.4467	0.0790	2.1717	0.0536
	MCGrad_group_features	0.0168	3.5723	0.0772	2.3468	0.0526
	MCGrad_no_unshrink	0.0164	3.4813	0.0796	2.2148	0.0536
	MCGrad_one_round	0.0162	3.4467	0.0790	2.1717	0.0536
	DFMC	0.0164	3.4734	0.0943	2.2036	0.0596
	HKRR	0.0180	3.7981	0.0559	2.5684	0.0538
	Isotonic	0.0168	3.5659	0.0847	2.3044	0.0544
BankMarketing	base_model	0.0195	7.0101	0.0522	5.1177	0.0565
	MCGrad_msh_20	0.0156	5.9140	0.0239	1.8957	0.0328
	MCGrad	0.0110	4.1535	0.0257	1.8693	0.0321
	MCGrad_group_features	0.0163	5.8165	0.0256	2.1236	0.0362
	MCGrad_no_unshrink	0.0103	3.8441	0.0222	2.1485	0.0317
	MCGrad_one_round	0.0139	5.0625	0.0384	3.0142	0.0415
	DFMC	0.0158	5.6785	0.0267	1.9211	0.0350
	HKRR	0.0270	9.1271	0.0360	3.9728	0.0390
	Isotonic	0.0177	6.3438	0.0288	2.4565	0.0370

Table 5: Multicalibration metrics.

dataset	mcb_algorithm	ECCE	ECCE/ σ	ECE	Log loss	PRAUC
MEPS	base_model	0.0157	2.3774	0.0215	0.3177	0.6211
	MCGrad_msh_20	0.0146	2.2479	0.0186	0.3150	0.6221
	MCGrad	0.0157	2.3774	0.0215	0.3177	0.6211
	MCGrad_group_features	0.0157	2.3774	0.0215	0.3177	0.6211
	MCGrad_no_unshrink	0.0157	2.3774	0.0215	0.3177	0.6211
	MCGrad_one_round	0.0140	2.1413	0.0189	0.3153	0.6204
	DFMC	0.0143	2.1821	0.0160	0.3163	0.6116
	HKRR	0.0166	2.5308	0.0252	0.3367	0.5317
	Isotonic	0.0112	1.7140	0.0118	0.3160	0.5945
acs_employment_all_states	base_model	0.0162	51.6562	0.0328	0.2257	0.9273
	MCGrad_msh_20	0.0010	3.8704	0.0013	0.1511	0.9597
	MCGrad	0.0012	4.4955	0.0023	0.1599	0.9512
	MCGrad_group_features	0.0008	2.5486	0.0009	0.2001	0.9391
	MCGrad_no_unshrink	0.0013	4.7459	0.0027	0.1588	0.9519
	MCGrad_one_round	0.0038	13.7094	0.0067	0.1698	0.9451
	DFMC	0.0011	3.7251	0.0025	0.2181	0.9349
	HKRR	0.0031	10.1073	0.0062	0.2111	0.9238
	Isotonic	0.0009	2.9845	0.0011	0.2066	0.9294
HMDA	base_model	0.0036	1.7643	0.0063	0.2761	0.9772
	MCGrad_msh_20	0.0044	2.3081	0.0057	0.2539	0.9818
	MCGrad	0.0048	2.5294	0.0060	0.2541	0.9818
	MCGrad_group_features	0.0029	1.4001	0.0030	0.2746	0.9775
	MCGrad_no_unshrink	0.0034	1.7673	0.0039	0.2546	0.9817
	MCGrad_one_round	0.0033	1.6494	0.0055	0.2609	0.9805
	DFMC	0.0032	1.5643	0.0055	0.2747	0.9775
	HKRR	0.0047	2.3059	0.0069	0.2773	0.9723
	Isotonic	0.0023	1.1215	0.0024	0.2757	0.9760
acs_travel_time_all_states	base_model	0.0093	10.4042	0.0181	0.6520	0.5258
	MCGrad_msh_20	0.0028	3.2589	0.0053	0.6304	0.5775
	MCGrad	0.0028	3.2067	0.0053	0.6303	0.5776
	MCGrad_group_features	0.0013	1.4963	0.0015	0.6449	0.5337
	MCGrad_no_unshrink	0.0021	2.4608	0.0037	0.6294	0.5781
	MCGrad_one_round	0.0053	5.9904	0.0068	0.6381	0.5579
	DFMC	0.0025	2.8485	0.0047	0.6460	0.5318
	HKRR	0.0051	5.7510	0.0052	0.6480	0.5083
	Isotonic	0.0032	3.5899	0.0007	0.6487	0.5233
acs_income_all_states	base_model	0.0067	9.7652	0.0135	0.4706	0.7407
	MCGrad_msh_20	0.0019	3.0261	0.0029	0.3924	0.8299
	MCGrad	0.0020	3.1717	0.0029	0.3924	0.8298
	MCGrad_group_features	0.0013	1.9035	0.0014	0.4608	0.7503
	MCGrad_no_unshrink	0.0012	1.9568	0.0018	0.3926	0.8296
	MCGrad_one_round	0.0034	5.2775	0.0049	0.4230	0.7963
	DFMC	0.0019	2.7820	0.0029	0.4648	0.7469
	HKRR	0.0032	4.7310	0.0038	0.4681	0.7230
	Isotonic	0.0015	2.1644	0.0018	0.4674	0.7402
ACSIIncome	base_model	0.0076	3.8599	0.0129	0.4635	0.7898
	MCGrad_msh_20	0.0064	3.7358	0.0120	0.3863	0.8599
	MCGrad	0.0065	3.8167	0.0114	0.3862	0.8600
	MCGrad_group_features	0.0019	1.0030	0.0046	0.4556	0.7963
	MCGrad_no_unshrink	0.0041	2.3710	0.0086	0.3853	0.8601
	MCGrad_one_round	0.0035	1.9324	0.0080	0.4076	0.8398
	DFMC	0.0019	0.9585	0.0045	0.4568	0.7955
	HKRR	0.0023	1.1808	0.0070	0.4604	0.7721
	Isotonic	0.0024	1.2285	0.0052	0.4605	0.7872
acs_health_insurance_all_states	base_model	0.0091	20.7777	0.0172	0.3971	0.2730
	MCGrad_msh_20	0.0011	2.5741	0.0019	0.3762	0.3331
	MCGrad	0.0011	2.5656	0.0018	0.3762	0.3334
	MCGrad_group_features	0.0005	1.1681	0.0008	0.3898	0.2933
	MCGrad_no_unshrink	0.0007	1.6650	0.0015	0.3762	0.3330
	MCGrad_one_round	0.0054	12.5842	0.0067	0.3844	0.3072
	DFMC	0.0018	4.1372	0.0055	0.3909	0.2912
	HKRR	0.0073	16.6308	0.0069	0.3944	0.2567
	Isotonic	0.0009	2.0126	0.0007	0.3951	0.2702
acs_public_health_insurance_all_states	base_model	0.0160	18.1469	0.0270	0.5194	0.5979
	MCGrad_msh_20	0.0013	1.6389	0.0022	0.4682	0.6684
	MCGrad	0.0014	1.7647	0.0021	0.4681	0.6686
	MCGrad_group_features	0.0018	2.0503	0.0024	0.5082	0.6091
	MCGrad_no_unshrink	0.0020	2.3991	0.0023	0.4672	0.6703
	MCGrad_one_round	0.0097	11.4243	0.0105	0.4934	0.6328
	DFMC	0.0024	2.7898	0.0029	0.5101	0.6056
	HKRR	0.0054	6.2067	0.0105	0.5147	0.5595
	Isotonic	0.0019	2.2277	0.0012	0.5144	0.5921
acs_mobility_all_states	base_model	0.0076	6.2231	0.0173	0.5540	0.8059
	MCGrad_msh_20	0.0054	4.6724	0.0080	0.5092	0.8694
	MCGrad	0.0053	4.6375	0.0079	0.5092	0.8695
	MCGrad_group_features	0.0021	1.6867	0.0036	0.5507	0.8112
	MCGrad_no_unshrink	0.0032	2.7402	0.0050	0.5092	0.8691
	MCGrad_one_round	0.0097	8.2706	0.0119	0.5176	0.8612
	DFMC	0.0029	2.4097	0.0035	0.5514	0.8095
	HKRR	0.0048	3.9479	0.0061	0.5532	0.7936
	Isotonic	0.0019	1.5359	0.0016	0.5521	0.8045
CreditDefault	base_model	0.0077	1.6218	0.0092	0.4333	0.5319
	MCGrad_msh_20	0.0056	1.1842	0.0103	0.4325	0.5320
	MCGrad	0.0056	1.1911	0.0110	0.4328	0.5318
	MCGrad_group_features	0.0058	1.2361	0.0118	0.4337	0.5291
	MCGrad_no_unshrink	0.0051	1.0907	0.0100	0.4327	0.5318
	MCGrad_one_round	0.0056	1.1911	0.0110	0.4328	0.5318
	DFMC	0.0059	1.2568	0.0129	0.4343	0.5275
	HKRR	0.0073	1.5338	0.0134	0.4402	0.4808
	Isotonic	0.0065	1.3702	0.0091	0.4445	0.5251
BankMarketing	base_model	0.0167	5.9821	0.0320	0.2433	0.5587
	MCGrad_msh_20	0.0034	1.2817	0.0062	0.2141	0.6190
	MCGrad	0.0031	1.1650	0.0051	0.2133	0.6209
	MCGrad_group_features	0.0035	1.2616	0.0039	0.2307	0.5637
	MCGrad_no_unshrink	0.0037	1.3911	0.0053	0.2137	0.6167
	MCGrad_one_round	0.0073	2.6721	0.0126	0.2255	0.5806
	DFMC	0.0035	1.2494	0.0059	0.2303	0.5634
	HKRR	0.0102	3.4546	0.0174	0.2453	0.4933
	Isotonic	0.0030	1.0791	0.0060	0.2312	0.5491

Table 6: (Global) Calibration and model performance metrics.